

Infografía

Módulo 12 – Godot 3D básico

Sesión 1: el espacio 3D

UPB · Gestión I/2026

Plan de la clase

1. **La idea:** casi todo lo de 2D tiene un **gemelo en 3D**.
2. **Node3D** y **Transform3D**: un eje más, rotación en tres ejes.
3. **Primitivas:** las mallas de fábrica.
4. **¿Por qué se ve negro?:** cámara + luz + entorno.
5. **Materiales PBR:** el "shader de fábrica" (eco del módulo 10).
6. **Transformar en código:** girar y orbitar (eco del módulo 6).
7. **Importar un modelo:** glTF y el *gotcha* de la escala.

Ya enviaste un juego 2D. Hoy: lo poco que cambia para hacerlo en 3D.

No empiezas de cero
Vienes a re-coordinar lo que ya sabes

El Rosetta 2D → 3D

Ya sabes (2D)	Gemelo en 3D
<code>Node2D</code>	<code>Node3D</code>
<code>Sprite2D</code>	<code>MeshInstance3D</code> + material
<code>Camera2D</code>	<code>Camera3D</code>
la luz era plana	<code>DirectionalLight3D</code> + <code>WorldEnvironment</code>
<code>CharacterBody2D</code>	cuerpo físico (sesión 2)
shader 2D (mód. 10)	material PBR
proyección a mano (mód. 6)	el motor multiplica las matrices

Lo nuevo de verdad: un eje más, la luz, y que el motor ya hace tu matemática.

¿Qué cambia? Un eje y una matriz más grande

En 2D un nodo tenía posición `(x, y)` y un ángulo.

En 3D un `Node3D` tiene un `Transform3D`: una `Basis` (3×3 , orientación + escala) y un `origin` (la posición `x, y, z`).

```
# 2D                                # 3D
position = Vector2(x, y)            position = Vector3(x, y, z)
rotation = a    # un ángulo         rotation = Vector3(rx, ry, rz)  # tres ejes
```

Es la misma matriz de transformación del módulo 6 — pero ahora la arma y la multiplica Godot.

Escena 01 — un Transform3D

Una caja **trasladada**, **rotada** y **escalada**, todo en una sola matriz.

Predice 🧠 — Muevo la caja a `position = (1.5, 0.5, 0)`.

¿Hacia dónde va: derecha y arriba, o derecha y *hacia el fondo*?

Y = arriba; Z = hacia/desde la cámara. El eje nuevo (Z) es la profundidad.

Escena 02 — primitivas

Las mallas que Godot trae de fábrica — el equivalente 3D de "dibujar una forma":

`BoxMesh` · `SphereMesh` · `CylinderMesh` · `CapsuleMesh` · `PlaneMesh`

Componer una escena = poner `MeshInstance3D` y moverlos en el espacio.

El `PlaneMesh` gigante de abajo es el piso. Todo lo demás se apoya en él.

■ ¿Por qué se ve todo **negro**?

El error más común al empezar en 3D

Pones tu malla, le das *play*... y la pantalla está vacía o negra. **Faltan dos cosas:**

1. Una **Camera3D** — sin un ojo, no hay imagen (en 2D la cámara era opcional).
2. **Luz** — sin luz que rebote, **toda superficie es negra**.

El **WorldEnvironment** agrega el cielo y la luz **ambiente** (relleno suave).

En 2D el sprite "se veía solo". En 3D, no hay color sin luz.

Escena 03 — súmalo de a uno (teclas 1–4)

- 1 — nada: superficies negras
- 2 — solo ambiente: plano, sin volumen
- 3 — solo sol: sombras duras, lados negros
- 4 — sol + ambiente: lo normal

`DirectionalLight3D` = el sol (rayos paralelos).

`ambient_light` = el relleno que evita las sombras 100% negras.

Mira el lado de la esfera opuesto al sol: con solo el sol, es negro; el ambiente lo "levanta".

Escena 04 — materiales PBR

La misma esfera, cinco `StandardMaterial3D`:

`albedo` (color) · `roughness` (rugosidad) · `metallic` (metalicidad) · `emission` (luz propia)

mate → satinado → brillante → metálico →
emisivo

En el módulo 10 **escribías** el shader.
Aquí ajustas parámetros y el motor arma el
sombreado físico por ti.

`roughness` *baja = brillo chico y duro.* `metallic` *alto = el color viene del reflejo.*

 En vivo

Transformar en código (escena **05**)

La luna ya orbita; el planeta no gira

Eco del módulo 6: ahí multiplicabas matrices a mano para rotar un punto. Aquí `rotate_y()` acumula una rotación frame a frame.

```
func _process(delta):  
    # ✅ dado: la luna orbita (giramos su pivote en el centro)  
    pivote_luna.rotate_y(deg_to_rad(giro_luna_grados) * delta)  
  
    # TODO (en vivo): que el planeta gire sobre su eje Y  
    pass
```

Estado roto: el planeta queda inmóvil mientras la luna ya orbita.

Escalera de pistas 🪜

1. **Qué** — el planeta tiene que girar sobre sí mismo, como la Tierra.
2. **Con qué** — el nodo `planeta` tiene el método `rotate_y(radianes)`.
3. **Casi la línea** — convierte grados→radianes y multiplica por `delta`:

```
planeta.rotate_y(deg_to_rad(giro_planeta_grados) * delta)
```

Predice 🧠 — Si borro `* delta`, ¿qué pasa a 30 FPS vs 144 FPS?

Sin `delta`, gira "por frame": más rápido cuanto más FPS. Con `delta`, gira por segundo: igual en cualquier máquina.

Escena 06 — importar un modelo glTF

Una primitiva la trae el motor; un **modelo** lo trae un archivo.

Estándar: **glTF** (`.gltf` texto o `.glb` binario). Trae de una vez **mall**a + **normales** + **materiales**. Godot lo importa como una **escena** (`PackedScene`).

⚠ El *gotcha*: **escala y orientación**. Cada herramienta exporta en su unidad y su eje. Si el modelo aparece gigante, minúsculo o de costado → casi siempre es eso.

Por eso dejamos al lado un cubo de 1×1×1: la regla para comparar el tamaño con el que entró la gema.

Cierre de la sesión 1

Ya sabes **ver y construir** en 3D:

- `Node3D` + `Transform3D`: posición, rotación (3 ejes), escala.
- Primitivas, cámara, **luz** (sin ella no hay color), entorno.
- Materiales PBR y transformar en código.
- Importar un modelo y vigilar la escala.

*Sesión 2: hacer que las cosas **se muevan** — física y un roll-a-ball jugable.*

¿Preguntas?

Siguiente: moverse en 3D 🎳